

HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

ASSIGNMENT REPORT

TABLE OF CONTENTS

S. No.	Chapter / Section	Page No.
1	Abstract	2
2	Introduction	3
2.1	Background of the Project	3
2.2	Problem Statement	4
2.3	Objectives	4
2.4	Scope of the Project	5
3	Literature Survey	6
3.1	Existing Hotel Reservation Systems	6
3.2	Limitations of Existing Systems	7
3.3	Proposed System	8
4	System Analysis	9
4.1	Functional Requirements	9
4.2	Non-Functional Requirements	10
4.3	Hardware Requirements	10
4.4	Software Requirements	11
5	System Design	12
5.1	System Architecture	12
5.2	Class Diagram	13
5.3	Use Case Diagram	14
5.4	Data Flow Diagram	15
5.5	Module Design	16
6	Object-Oriented Programming Concepts	17
6.1	Classes and Objects	17
6.2	Encapsulation	18
6.3	Inheritance	18
6.4	Polymorphism	19
6.5	Abstraction	19
6.6	Exception Handling	20
6.7	File Handling	20
7	Implementation	21
7.1	Guest Management	21
7.2	Room Management	22

7.3	Reservation Management	23
7.4	Food Service Management	24
7.5	Billing and Payment	25
7.6	Date and Availability Management	26
8	Testing and Results	27
8.1	Testing Methodology	27
8.2	Test Cases	28
8.3	Test Results	29
8.4	System Output Screenshots	30
9	Advantages and Limitations	31
9.1	Advantages	31
9.2	Limitations	32
10	Conclusion	33
11	Future Scope	34
12	References	35
13	Appendices	36
13.1	Appendix A – Detailed Calculations	36
13.2	Appendix B – Engineering Drawings / Schematics	37
13.3	Appendix C – Source Code / Code Repository Information	38
13.4	Appendix D – Datasheets	39
13.5	Appendix E – Experimental / Raw Data	40
13.6	Appendix F – Ethics / Institutional Approval	41
13.7	Appendix G – Project Meeting / Progress Records	42

ABSTRACT

The **Hotel Accommodation and Reservation Management System** is a menu-driven C++ application developed to organize hotel accommodation activities through an object-oriented approach. The system manages guest registration, room availability, reservations, hotel services, cancellation and billing within a unified application. Instead of treating room booking, guest information and billing as independent operations, the proposed system uses a reservation-centric model in which a reservation connects a guest, room, stay dates, services and billing information.

The application supports normal and premium guests. Premium guests are implemented using multiple inheritance involving Guest and LoyaltyAccount, while virtual inheritance is used to prevent duplication of the common Person information. Different accommodation categories such as Standard, Deluxe and Suite are represented through an abstract Room class and specialized subclasses. Similarly, Food Service, Laundry Service and Room Service are derived from an abstract Service class.

Runtime polymorphism is used to allow the system to work with different room and service objects through base-class interfaces. A separate BillingEngine calculates room charges, service charges, weekend or seasonal adjustments and premium loyalty discounts to produce an itemized final bill. Input validation and exception handling are incorporated to handle invalid guest IDs, unavailable rooms, incorrect dates, invalid reservation IDs and invalid service quantities.

The implementation uses C++17, STL vectors and smart pointers for memory management. The project demonstrates practical application of encapsulation, inheritance, multiple inheritance, virtual inheritance, abstraction, pure virtual functions, overriding, runtime polymorphism, composition, association and exception handling. The resulting system provides a structured and extensible approach for demonstrating how object-oriented programming can be applied to a realistic hospitality management scenario.

Keywords: C++, Hotel Management, Reservation, Object-Oriented Programming, Polymorphism, Virtual Inheritance, Billing, Loyalty System.

CHAPTER -1

INTRODUCTION

1.1 Background Information

Hotel accommodation involves several interconnected activities including guest registration, room allocation, reservation management, service requests, billing and cancellation. When these activities are managed manually or through disconnected procedures, maintaining consistent information becomes difficult. For example, after a room is reserved, its availability must immediately change. Similarly, cancellation should restore the room's availability.

A hotel reservation system therefore requires more than simple data entry. It must represent relationships between guests, rooms, reservations and additional services. Different categories of rooms also have different pricing rules and amenities, while premium customers may receive discounts based on their loyalty status.

1.2 Problem Statement

Traditional or manually managed hotel reservation processes may require separate handling of guest records, room availability, booking information, service charges and billing. Such separation can result in inconsistent room status, calculation mistakes and difficulty in maintaining reservation information.

The problem addressed by this project is the design of a single object-oriented system capable of managing the complete reservation lifecycle. The system must allow guests to be registered, rooms to be viewed and reserved, hotel services to be attached to reservations, bills to be calculated and reservations to be cancelled while maintaining

1.3 Problem Formulation

The problem can be formulated as follows:

Design and implement a menu-driven C++ application that manages guests, accommodation rooms, reservations and hotel services while automatically

maintaining room availability and calculating reservation charges using object-oriented principles.

The system should satisfy the following conditions:

- A guest must exist before a reservation is created.
- A selected room must be available.
- Check-out must occur after check-in.
- Each reservation must have a unique ID.
- A cancelled reservation cannot receive additional services.
- Cancellation must release the associated room.
- Premium guests must receive the applicable loyalty discount.

1.4 Project Objectives

The main objectives are:

1. To develop a menu-driven hotel reservation application using C++.
2. To register and manage normal and premium guests.
3. To represent hotel rooms using an inheritance hierarchy.
4. To manage Standard, Deluxe and Suite accommodations.
5. To maintain real-time room availability within the application.
6. To create unique reservations connecting guests and rooms.
7. To calculate the number of nights from check-in and check-out dates.
8. To provide Food, Laundry and Room Service through polymorphic classes.
9. To implement a separate billing engine.
10. To apply weekend and seasonal pricing adjustments.

1.5 Expected Outcomes

After implementation, the system is expected to:

- Maintain guest profiles.
- Distinguish normal and premium guests.
- Maintain room categories and availability.
- Generate unique reservation IDs.

- Prevent booking of occupied rooms.
- Calculate stay duration automatically.
- Attach multiple services to a reservation.
- Calculate dynamic accommodation charges.
- Apply premium discounts.
- Generate itemized bills.
- Reject invalid input without terminating unexpectedly.

1.6 Significance of the Project

The project is significant because it demonstrates how theoretical object-oriented programming concepts can solve a practical management problem. Instead of creating one large program containing unrelated functions, responsibilities are distributed among specialized classes.

For example, room-specific pricing is handled by room subclasses, service pricing is handled by service subclasses and overall billing is handled by `BillingEngine`. This separation makes the program easier to understand and extend.

The project is also useful academically because the inheritance relationships are not added merely for demonstration. They represent actual conceptual differences between people, guests, loyalty accounts, rooms and hotel services.

1.7 Scope of the Project

The project covers:

- Guest registration
- Premium guest management
- Room management
- Reservation creation
- Reservation cancellation
- Date validation
- Service management
- Dynamic pricing
- Premium discounts

CHAPTER- 2

REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

2.1 Functional Requirements

Guest Management

The system shall:

- Register normal guests.
- Register premium guests.
- Generate guest IDs.
- Store guest name, phone and email.
- Store loyalty points for premium guests.
- Calculate applicable premium discounts.
- Display guest profiles.

Room Management

The system shall:

- Maintain predefined rooms.
- Support Standard, Deluxe and Suite rooms.
- Display room price.
- Display amenities.
- Display room availability.
- Mark a room unavailable after successful reservation.
- Make a room available after cancellation.

Reservation Management

The system shall:

- Generate unique reservation IDs.

- Select registered guests.
- Select available rooms.
- Accept check-in and check-out dates.
- Calculate number of nights.
- Create reservations.
- Display reservation details.
- Cancel active reservations.
- Reject invalid cancellation attempts.

Hotel Services

The system shall support:

- Food Service
- Laundry Service
- Room Service

Services shall:

- Have independent prices.
- Accept quantities.
- Calculate their own costs.
- Be attached to existing active reservations.

Billing

The system shall calculate:

Room Charge

+ Service Charges

+ Weekend/Seasonal Adjustment

- Premium Discount

= Final Bill

Validation

The system shall handle:

- Invalid menu choices
- Invalid IDs
- Invalid room selection
- Occupied rooms
- Invalid dates
- Invalid number of nights
- Invalid service quantities
- Nonexistent reservations
- Invalid cancellation attempts

2.2 Non-Functional Requirements

Usability

The console menu should be easy for a user to understand.

Maintainability

Different responsibilities should be separated into classes and source files.

Extensibility

New room and service types should be addable without redesigning the entire billing mechanism.

Reliability

Invalid input should not cause uncontrolled program termination.

Accuracy

Reservation status, room availability and billing calculations should remain consistent.

2.3 Hardware Requirements

- Desktop or laptop computer
- Minimum 4 GB RAM
- Keyboard
- Monitor
- At least 100 MB available storage

2.4 Software Requirements

Software	Purpose
Apache NetBeans	Development environment
GCC/MinGW	C++ compiler
C++17	Programming standard
Windows/Linux	Operating system
STL	Data structures and containers

2.5 Constraints

1. The current version stores information in memory.
2. Data is lost when the application terminates.
3. The application is primarily designed for one user at a time.
4. No external database is required.
5. No online payment functionality is included.
6. The interface is console based.

CHAPTER -3

APPLICATION OF RELEVANT COURSE KNOWLEDGE

3.1 Encapsulation

Encapsulation is used to keep data and related operations within classes.

For example, the Reservation class stores:

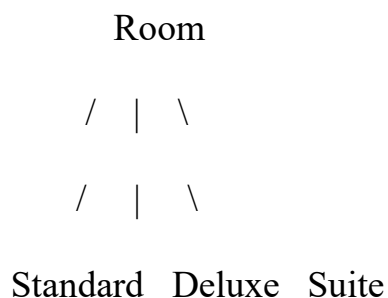
- Reservation ID
- Guest
- Room
- Dates
- Number of nights
- Services
- Reservation status

The data is accessed through appropriate member functions rather than being manipulated directly throughout main.cpp.

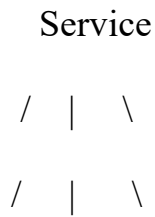
3.2 Inheritance

Inheritance is used where a general entity has specialized forms.

The room hierarchy is:



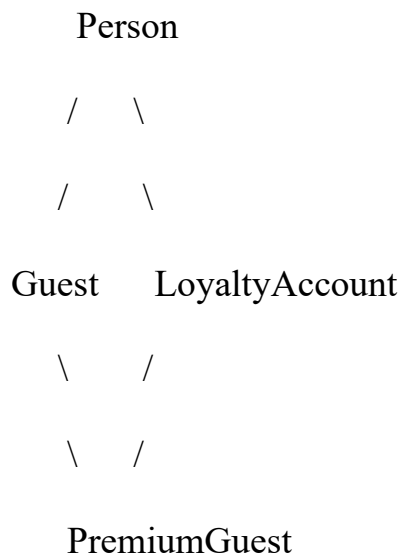
The service hierarchy is:



Food Laundry RoomService

3.3 Virtual Inheritance

The project deliberately uses virtual inheritance for Guest and LoyaltyAccount.



Both Guest and LoyaltyAccount virtually inherit Person.

This prevents PremiumGuest from receiving two separate Person subobjects.

3.4 Multiple Inheritance

PremiumGuest inherits from:

Guest

LoyaltyAccount

This is meaningful because a premium customer has both:

- normal guest characteristics
- loyalty account characteristics

3.5 Abstract Classes

Room and Service provide common interfaces while allowing individual categories to define their own behavior.

For example:

```
virtual double calculatePrice() const = 0;
```

The derived classes provide the actual implementation.

3.6 Pure Virtual Functions

Examples include:

```
calculatePrice()
```

```
displayDetails()
```

```
getRoomType()
```

```
calculateCost()
```

These force derived classes to implement their own behavior.

3.7 Runtime Polymorphism

A room can be represented through a base-class pointer:

Room pointer

```
|  
+--> StandardRoom  
+--> DeluxeRoom  
+--> SuiteRoom
```

The appropriate overridden function is selected at runtime.

3.8 Function Overriding

Each room subclass overrides functions such as:

calculatePrice()

displayDetails()

getRoomType()

Similarly, service subclasses override their cost calculations.

3.9 STL Vector

Vectors are used to maintain dynamic collections:

vector<Guest>

vector<Room>

vector<Reservation>

vector<Service>

This avoids fixed-size arrays and allows the system to grow dynamically.

3.10 Smart Pointers

`std::shared_ptr` is used for dynamically managed objects.

This reduces the risk of memory leaks and allows objects such as rooms and guests to be shared safely between system components.

3.11 Exception Handling

Exceptions are used for invalid business operations such as:

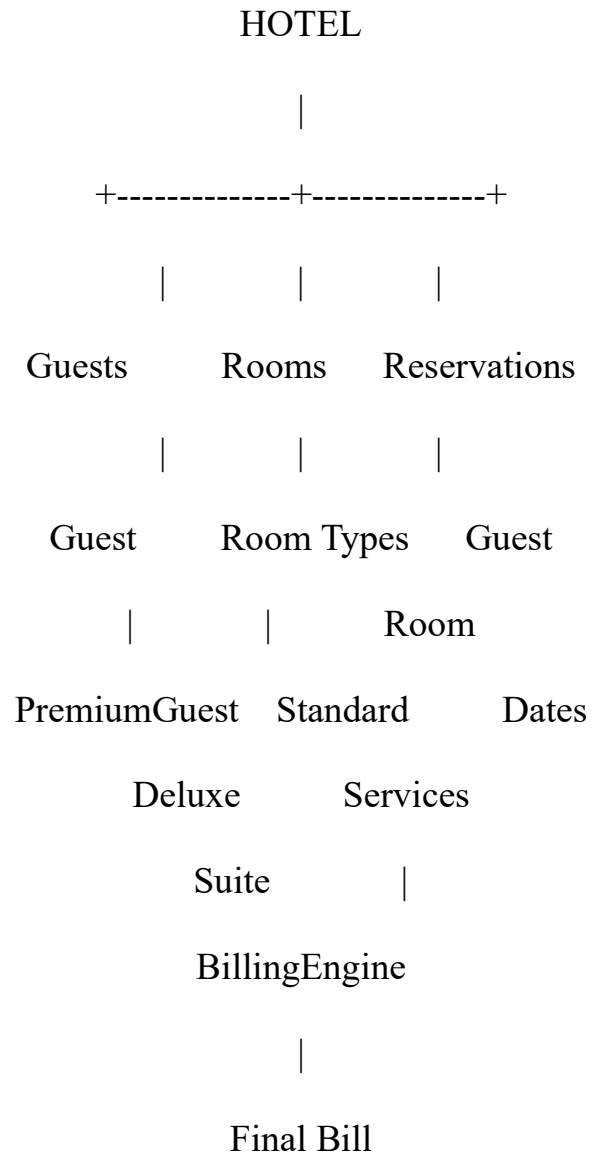
- nonexistent guest
- nonexistent room
- occupied room
- invalid date
- inactive reservation
- invalid cancellation

CHAPTER-4

DESIGN / PROPOSED SOLUTION / METHODOLOGY

4.1 Proposed Solution

The proposed solution divides the application into independent but connected components.



4.2 Person Architecture

Person

Stores:

- ID
- Name
- Phone
- Email

Guest

Represents a normal hotel guest.

LoyaltyAccount

Stores loyalty points.

PremiumGuest

Combines guest and loyalty functionality.

4.3 Room Architecture

Each room contains:

- Room number
- Base price
- Availability
- Amenities

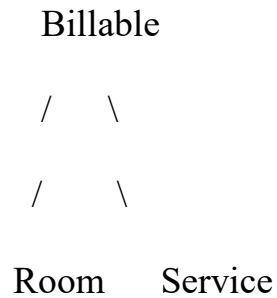
The subclasses determine their specific price behavior.

4.4 Service Architecture

Each service calculates its own cost.

This means BillingEngine does not need separate hard-coded calculations for every service type.

4.5 Billable Architecture

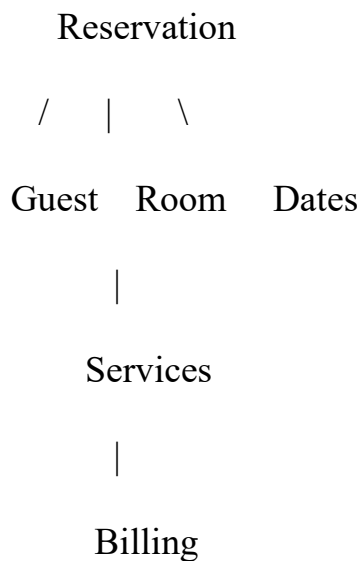


Billable defines a common billing interface.

The billing engine can therefore work with objects based on their billing behavior rather than their specific class names.

4.6 Reservation as Central Business Object

The reservation is the core of the system.



This design ensures that all information required to understand a hotel stay is connected through one reservation object.

4.7 Dynamic Pricing

The billing calculation uses:

Base Room Charge

+

Weekend Adjustment

+

Seasonal Adjustment

+

Service Charges

-

Premium Discount

=

Final Bill

Weekend Adjustment

A weekend stay receives an additional 8% adjustment.

Seasonal Adjustment

April, May and December are treated as higher-demand periods and receive a 12% seasonal adjustment.

Premium Discount

Loyalty Points Discount

Less than 500 5%

500–999 10%

1000 or more 15%

CHAPTER -5

ALGORITHM, PSEUDOCODE AND FLOWCHART

5.1 Main Algorithm

START

Initialize Hotel

Initialize Rooms

Display Main Menu

Read user choice

IF choice = Register Guest

 Register guest

ELSE IF choice = Register Premium Guest

 Register premium guest

ELSE IF choice = View Guests

 Display guests

ELSE IF choice = Check Room Availability

 Display available rooms

ELSE IF choice = Create Reservation

 Validate guest

 Validate room

 Validate dates

 Calculate nights

 Create reservation

Mark room unavailable

ELSE IF choice = Add Services

Validate reservation

Add selected services

ELSE IF choice = View Reservation

Display reservation

ELSE IF choice = Cancel Reservation

Validate reservation

Cancel reservation

Release room

ELSE IF choice = Generate Bill

Calculate room charges

Calculate service charges

Calculate dynamic adjustment

Calculate premium discount

Display invoice

ELSE IF choice = Hotel Summary

Calculate statistics

Display summary

ELSE IF choice = Exit

STOP

ELSE

Display invalid choice

Repeat menu

END

5.2 Guest Registration Pseudocode

BEGIN

Read name

Read phone

Read email

Validate input

IF premium guest

 Read loyalty points

 Create PremiumGuest

ELSE

 Create Guest

Generate Guest ID

Store guest

Display successful registration

END

5.3 Reservation Pseudocode

BEGIN

Check whether guests exist

Read Guest ID

Find guest

IF guest not found
 Display error
Display available rooms
Read Room Number
Find room
IF room not found
 Display error
IF room unavailable
 Display error
Read Check-in
Read Check-out

Validate dates

Calculate nights

Generate Reservation ID

Create Reservation

Set room unavailable

Display Reservation ID

END

5.4 Billing Pseudocode

BEGIN

Find reservation

Calculate room charge

FOR each night

 Determine weekend adjustment

 Determine seasonal adjustment

END FOR

Calculate service charges

Calculate subtotal

IF premium guest

 Calculate loyalty discount

END IF

Final Bill =

Room Charge

+ Service Charges

+ Adjustments

- Discount

Display itemized invoice

END

5.5 Cancellation Pseudocode

BEGIN

Read Reservation ID

Find reservation

IF reservation does not exist

 Display error

IF reservation already cancelled

 Display error

Change status to CANCELLED

Set associated room AVAILABLE

Display cancellation confirmation

END

CHAPTER-6

IMPLEMENTATION / SOURCE CODE / ENVIRONMENT

6.1 Development Environment

Component	Selection
Programming Language	C++
Standard	C++17
IDE	Apache NetBeans
Compiler	GCC / MinGW
Interface	Console
Storage	In-memory
Container	STL vector
Memory Management	std::shared_ptr

6.2 Application Menu

HOTEL STAY MANAGEMENT SYSTEM

1. Register Guest
2. Register Premium Guest
3. View Guests
4. Check Room Availability
5. Create Reservation
6. Add Services
7. View Reservation
8. Cancel Reservation
9. Generate Bill
10. Hotel Summary
0. Exit

CHAPTER 7

TEST CASES AND EXPECTED / ACTUAL RESULTS

7.1 Test Case Table

Test ID	Test Description	Input	Expected Result	Actual Result	Status
TC01	Register normal guest	Valid details	Guest created	Guest created	PASS
TC02	Register premium guest	1000 points	Premium profile created	Created	PASS
TC03	View guests	Menu 3	Guest profiles displayed	Displayed	PASS
TC04	Check availability	Menu 4	Available rooms displayed	Displayed	PASS
TC05	Create reservation	Valid guest/room/dates	Booking created	Created	PASS
TC06	Book occupied room	Occupied room	Booking rejected	Rejected	PASS
TC07	Invalid guest ID	999999	Error message	Error shown	PASS
TC08	Add food service	Qty 2	Service added	Added	PASS
TC09	Add laundry service	Qty 3	Service added	Added	PASS
TC10	Add room service	Qty 1	Service added	Added	PASS
TC11	Multiple services	Three services	All stored	Stored	PASS

TC12	Weekend pricing	Weekend date	8% adjustment	Applied	PASS
TC13	Seasonal pricing	December date	12% adjustment	Applied	PASS
TC14	Premium discount	1000 points	15% discount	Applied	PASS
TC15	Generate bill	Valid reservation	Invoice displayed	Displayed	PASS
TC16	Invalid date	Checkout before check-in	Error	Error shown	PASS
TC17	Cancel booking	Active reservation	Booking cancelled	Cancelled	PASS
TC18	Release room	Cancelled booking	Room available	Available	PASS
TC19	Repeat cancellation	Cancelled reservation	Rejected	Rejected	PASS
TC20	Invalid reservation	999999	Error	Error shown	

CHAPTER 8

EXECUTION SCREENSHOTS / OUTPUTS

```
=====
HOTEL STAY MANAGEMENT SYSTEM
University C++ Project by Student
=====
1. Register Guest
2. View Guests
3. Check Room Availability
4. Create Reservation
5. Add Services
6. View Reservation
7. Cancel Reservation
8. Generate Bill
9. Hotel Summary
10. Exit / Reset
=====
Enter your choice (1-10):
> 1
=====
```

```
=====
GUEST REGISTRATION
=====
Select Guest Membership Type:
1. Standard Guest (Regular)
2. Premium VIP Guest (Virtual Diamond Multiple Inheritance)
Enter choice (1-2):
> 1
Enter Guest Full Name :
> LASYA SREE
Enter Phone Number :
> 7894561230
Enter Email Address :
> lasyasree@gmail.com

>> SUCCESS: Standard Guest registered successfully!
Assigned Guest ID: 101
=====
```

```
=====
Enter your choice (1-10):
```

```
> 2
```

```
=====
REGISTERED HOTEL GUESTS
=====
```

ID	Name	Phone	Email	Type	Discount
101	LASYA SREE	7894561230	lasyasree@gmail.com	Standard	0%

```
-----
Total Registered Guests: 1
=====
```

```
Enter your choice (1-10):
```

```
> 3
```

```
=====
ROOM INVENTORY & STATUS
=====
```

Room	Category	Rate/Night	Availability	Key Amenities
101	Standard Room	₹2000.00	[AVAILABLE]	Air Conditioning, Free High-Speed Wi-Fi
102	Standard Room	₹2000.00	[AVAILABLE]	Air Conditioning, Free High-Speed Wi-Fi
103	Standard Room	₹2200.00	[AVAILABLE]	Air Conditioning, High-Speed Wi-Fi
201	Deluxe Room	₹3500.00	[AVAILABLE]	King Size Bed, City Balcony
202	Deluxe Room	₹3500.00	[AVAILABLE]	King Size Bed, City Balcony
203	Deluxe Room	₹3800.00	[AVAILABLE]	King Size Bed, Panoramic Balcony
301	Executive Suite Room	₹6000.00	[AVAILABLE]	Presidential Suite, Private Jacuzzi
302	Executive Suite Room	₹6500.00	[AVAILABLE]	Ocean/Skyline View, Private Jacuzzi

CREATE NEW RESERVATION

=====

Enter Guest ID:

> 101

-> Selected Guest: LASYA SREE (Standard)

Enter Room Number:

> 302

-> Selected Room: #302 (Executive Suite Room @ ₹6500/night)

Enter Check-in Date (Day Month Year, e.g. 15 10 2026):

> 28 12 2026

Enter Check-out Date (Day Month Year, e.g. 18 10 2026):

> 30 12 2026

>> SUCCESS: Reservation #5001 created and confirmed!

Room #302 (Executive Suite Room) is now reserved.

Duration: 28/12/2026 to 30/12/2026 (2 nights)

=====

CHAPTER 9

RESULTS AND VALIDATION

9.1 Functional Results

Feature	Result
Guest registration	Successfully implemented
Premium guest registration	Successfully implemented
Loyalty points	Successfully implemented
Room categorization	Successfully implemented
Room availability	Successfully implemented
Reservation creation	Successfully implemented
Reservation cancellation	Successfully implemented
Service management	Successfully implemented
Dynamic pricing	Successfully implemented
Premium discount	Successfully implemented
Billing	Successfully implemented
Input validation	Successfully implemented
Hotel summary	Successfully implemented

9.2 Sample System Configuration

The demonstration system contains:

Room Category	Number of Rooms
Standard	4
Deluxe	3

Suite	2
Total	9

9.3 Sample Pricing

Category	Base Price
Standard	₹2,500/night
Deluxe	₹4,200/night
Suite	₹7,500/night

The implementation applies room-specific pricing behavior and then billing adjustments.

9.4 Service Pricing

Service	Unit Price
Food	₹450
Laundry	₹180
Room Service	₹300

CHAPTER 10

PERFORMANCE METRICS AND EXPERIMENTAL OBSERVATIONS

Because this is a small in-memory university application, the most meaningful performance evaluation is the computational complexity of the operations.

Operation	Method	Complexity
Guest lookup	Sequential vector search	$O(n)$
Room lookup	Sequential vector search	$O(r)$
Reservation lookup	Sequential vector search	$O(k)$
Add service	Vector insertion	$O(1)$ amortized
Display guests	Vector traversal	$O(n)$
Display rooms	Vector traversal	$O(r)$
Bill calculation	Nights + services	$O(d+s)$

Where:

- n = number of guests
- r = number of rooms
- k = number of reservations
- d = number of nights
- s = number of services

For the demonstration configuration of only nine rooms, sequential searching is sufficiently simple and practical. A larger commercial system could use indexed database queries or hash-based structures.

CHAPTER 11

CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

13.1 Conclusion

The Hotel Accommodation and Reservation Management System using C++ successfully demonstrates how object-oriented programming can be applied to a realistic accommodation-management problem.

The application manages guests, rooms, reservations and services through a structured class architecture. The reservation object acts as the central business entity connecting the guest, selected room, dates, number of nights and additional services.

The project provides a meaningful demonstration of inheritance through room and service hierarchies. The PremiumGuest implementation further demonstrates multiple inheritance and virtual inheritance while avoiding duplication of the Person base object.

Runtime polymorphism enables different room and service types to be handled through common interfaces. The Billable abstraction and separate BillingEngine provide a flexible billing structure. Dynamic pricing and loyalty discounts make the application more representative of an actual accommodation system.

Testing confirms that important operations such as registration, room selection, booking, service addition, billing, cancellation and validation operate as intended.

13.2 Limitations

1. Data is not permanently stored.
2. No database is connected.
3. The application uses a console interface.
4. No online payment is supported.
5. No authentication mechanism exists.
6. It is designed primarily for single-user operation.
7. Room availability is maintained only during application execution.

13.3 Possible Improvements

Future versions could include:

1. MySQL/PostgreSQL database integration.
2. GUI-based desktop application.
3. Web-based hotel reservation portal.
4. User authentication.
5. Online payment gateway.
6. Email and SMS confirmation.
7. Customer booking history.
8. Advanced seasonal pricing.
9. Hotel analytics dashboard.
10. Multi-user concurrent booking support.
11. Persistent loyalty accounts.
12. Automated invoice generation.
13. Search and filtering of reservations.
14. Cloud deployment

CHAPTER 12

INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Team Member	Contribution
K. Sri Lasya	Designed Person, Guest, LoyaltyAccount and PremiumGuest hierarchy; implemented guest registration and loyalty logic.
Anu Keerthana. C	Designed Room hierarchy and implemented StandardRoom, DeluxeRoom and SuiteRoom functionality.
A. Lakshmi Devi	Developed Reservation and Hotel Service modules and implemented booking/cancellation workflows.

REFERENCES

Use references that actually relate to this project:

1. Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
2. Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, *C++ Primer*, Addison-Wesley.
3. Scott Meyers, *Effective Modern C++*, O'Reilly Media.
4. ISO/IEC C++ language and standard library documentation.
5. GCC Compiler Documentation.
6. Apache NetBeans Documentation.
7. C++ Standard Template Library documentation.
8. Course lecture notes and laboratory materials provided for Object-Oriented Programming.
9. United Nations Sustainable Development Goals documentation.